

## Project Report: The Orthogonal writing mech.

**Competition:** Build-a-Bot 3.0, Round 1 **Organization:** PARSEC, IIT Dharwad

### 1. Problem Approach

The team's primary objective was to design and simulate a 2-Degree-of-Freedom (2-DoF) planar writing mechanism capable of real-time movement via ROS2 teleoperation.

- **Iterative Design:** Our initial approach involved a planar arm configuration using two rotary joints. However, to prioritize precision and simplify the mapping between Cartesian control commands and joint motion, the team pivoted to a **Cartesian (Orthogonal) design**.
  - **Workflow:**
    1. **Modelling:** Developed a 2-DoF orthogonal mechanism in the simulation environment.
    2. **Environment Setup:** Configured a ROS2 Jazzy environment within Windows Subsystem for Linux (WSL) to leverage Linux-based robotics tools on a Windows host.
    3. **Bridge Implementation:** Established a communication link between ROS2 and the simulator (CoppeliaSim/Gazebo) to pass control signals.
    4. **Control Integration:** Integrated the standard teleop\_twist\_keyboard package to drive the mechanism.
- 

### 2. System Architecture & Design Choices

The system architecture is divided into three main layers: the Input Layer, the Control Logic Layer, and the Simulation Layer.

#### Mechanical Design: Orthogonal Frame

The team selected the **Cartesian design** over the planar arm for the following reasons:

- **Linear Mapping:** Movement along the X and Y axes maps directly to individual linear joints, eliminating the need for complex inverse kinematics required by rotary arms.
- **Stability:** Orthogonal frames typically offer higher rigidity for writing tasks, ensuring "fluid motions" as requested in the brief.

#### Software Stack

- **Operating System:** ROS2 Jazzy running on WSL.
- **Simulator:** CoppeliaSim.
- **Communication:** A ROS2 node subscribes to the /cmd\_vel topic (type: geometry\_msgs/Twist).
  - linear.x commands are translated to the X-axis actuator.
  - linear.y commands are translated to the Y-axis actuator.

### 3. Assumptions and Limitations

#### Assumptions

- **Idealized Friction:** The simulation assumes negligible friction between the pen tip and the writing surface to ensure "smooth bidirectional motion"
- **Constant Velocity:** It is assumed that the teleop\_twist\_keyboard provides a steady velocity stream which the joint controllers can follow without significant lag

#### Limitations

- **WSL Latency:** Running ROS2 through WSL can occasionally introduce minor communication latency between the ROS2 node and the simulator compared to a native Linux installation.
- **2-DoF Constraints:** As a 2-DoF system, the "Virtual Scribe" cannot lift the pen (Z-axis). It remains in constant contact with the writing surface, meaning all movements result in a continuous stroke.
- **Manual Control Only:** The current implementation relies strictly on manual keyboard teleoperation and does not yet include automated path planning or G-code parsing.